# Learning Image Query Concepts via Intelligent Sampling

Beitao Li
beitao@engineering.ucsb.edu

Edward Chang
echang@ece.ucsb.edu

Chung-Sheng Li
csli@us.ibm.com

## Abstract

In this paper, we propose an *active* and *inductive* combined learning method to learn users' image query concepts. We model query concepts in $k$-CNF, which can be used to express most practical queries. To learn a user's query concept, we propose MEGA. MEGA initializes a user's query concept as the conjunction of all disjunctions of at most length $k$ of the predicates. It then intelligently selects unlabeled data to present to the user for gathering information to eliminate the maximum expected number of disjunctions. MEGA maximizes the usefulness of each example it generates for learning a user's query concept and hence expedites the convergence to the target concept. Through analysis and experiments, we show that MEGA can learn a complex image query concept much faster than some traditional schemes.

## 1 Introduction

For many search tasks, a query concept is hard to articulate, and articulation can be subjective. For instance, in image queries, it is difficult to describe desired images using low-level features such as color, shape and texture (these are widely used features for representing images [8]), and different users may use different combinations of these features to depict the same image. In this paper, we propose an *inductive* and *active* combined learning method that learns users' image query concepts via intelligent sampling. Our method converges to the target query concept significantly faster than traditional query refinement schemes.

Inductive learning finds a query concept through labeled data, and active learning assists the learning process to converge to the concept quickly. Inductive learning faces two challenges. First, the version space $V$ is potentially exponential in the size of hypotheses [6]. To tackle the large-version-space problem, we limit our search engine to learn $k$-CNF concepts. We believe that almost all practical query concepts can be modeled by $k$-CNF, which can be learned in polynomial time using positive-labeled instances [6]. The second challenge is that traditional inductive learning methods, such as neural networks and decision trees, require a large set of labeled instances to learn a concept. Unfortunately, since on-line users are typically impatient and cannot be expected to provide much feedback (i.e., to provide a large number of labeled instances), traditional inductive schemes are not suitable for on-line learning.

In contrast to such existing learning methods, we propose employing an intelligent sampling scheme that can quickly gather maximum information for learning users' query concepts. In other words, our learner maximizes the usefulness of each sample it generates to query the user and therefore requires a far smaller number of samples to learn the target concept than traditional sampling schemes require. Our proposed scheme—MEGA (the Maximizing Expected Generalization Algorithm)—judiciously selects samples at each iteration and uses positive-labeled instances to learn the target concept in $k$-CNF. At the same time, it uses negative-labeled instances to learn a $k$-DNF that bounds the uncertain region, in order to select the most informative samples. Our experiments using both synthesized data and real images show that MEGA converges to a target concept much faster than some traditional schemes (e.g., MARS).

### 1.1 Related Work

In most inductive learning problems studied in the AI community, samples are assumed to be taken randomly in such a way that various statistical properties can be derived conveniently. However, for interactive applications where the number of samples must be small (or impatient users might be turned away), random sampling is not suitable.

Relevance feedback techniques proposed by the IR (Information Retrieval) community do perform non-random sampling [5]. For instance, MARS [7] employs IR techniques that move the query point in the feature space and also weight features differently according to users' feedback. Traditional IR techniques, however, tend to refine a search only in the nearest neighborhood. By the definition of nearest neighbors, performing this kind of query refinement cannot achieve significant improvement on the query, especially when a user does not begin the search with a good query object (or objects).

Of late, pool-based learning or active learning has been introduced for choosing good samples from the unlabeled data pool to speed up the learning process. For image retrieval, the PicHunter system [3] uses Bayes' rule to predict the goal image, based upon the users' actions. The system shows that employing active learning can drastically cut down the number of iterations (up to $80\%$ in some experiments). But, the authors also point out that their scheme is computationally intensive since it recomputes conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with dataset size.

Another method, the Query by Committee (QBC) algorithm [4], employs a committee of learners to select unlabeled samples that yield maximum information gain. Two major challenges of applying the QBC algorithm to a learning task are 1) finding useful committee members, and 2) finding informative unlabeled instances efficiently. Our proposed MEGA can be regarded as a QBC algorithm. MEGA provides an effective method for refining committee members (i.e., a $k$-CNF and a $k$-DNF hypothesis), and at the same time delimits the boundary of the sampling space for efficiently finding useful samples. MEGA also works well with moderately noisy feedback that

early QBC schemes [4] were not designed to deal with.

## 2 Learning Query Concepts

Our learning algorithm consists of two major parts:

- Refinement. Refine the query concept based on positive-labeled instances. The refinement step is carefully designed to tolerate noisy data.
- Sampling. Refine the sampling space based on negative-labeled instances and select samples judiciously for expediting the learning process.

### 2.1 Refinement

We make significant changes to Valiant's algorithm [10] to

- Handle the fuzzy membership functions (described in [2]).
- Tolerate user errors (Section 2.3), and
- More importantly, select samples judiciously to expedite the learning process (Section 2.2).

In our method, the concept learner initializes the query concept space ($QCS$) as a $k$-CNF and the candidate concept space ($CCS$) as a $k$-DNF. The $QCS$ starts as the most specific concept and the $CCS$ as the most general concept. The target concept that the concept learner learns is more general than the initial $QCS$ and more specific than the initial $CCS$. The concept learner learns the $QCS$, while at the same time refining the $CCS$ to delimit the boundary of the sampling space. The learning algorithm can be summarized as the following five steps:

1. Generate samples from the unlabeled pool $U$. This step passes $QCS$, $CCS$ and $U$ to procedure *MEGA* to generate $K_\alpha$ samples. We will discuss procedure *MEGA* in Section 2.2.

2. Solicit user feedback. A user marks an object positive if the object fits his/her query concept. An unmarked object is considered negative.

3. Refine $QCS$. For each disjunctive term in the $QCS$, the learner examines each positive-labeled instance and uses function $f$ to compute the probability that the disjunctive term should be eliminated. The learner then calls procedure *Vote* (We discuss procedure *Vote*, which decides how aggressive the learner is in eliminating terms, in Section 2.3.) to tally the votes among the positive-labeled instances and compares the vote with threshold $\gamma$ to decide whether that disjunctive term should be removed. If sufficient numbers of positive-labeled instances contradict the $QCS$ with respect to a disjunctive term (i.e., if the threshold is exceeded), the term is removed from the $QCS$.

4. Refine $CCS$. Similarly, for each conjunctive term in the $CCS$, the learner examines each negative-labeled instance and uses function $f$ to compute the probability that the conjunctive term should be eliminated. The learner then calls procedure *Vote* to tally the votes among the negative-labeled instances and then compares the vote with threshold $\gamma$ to decide whether that conjunctive term should be removed. If sufficient numbers of negative-labeled instances satisfy the $CCS$ with respect to a conjunctive term, the term is removed from the $CCS$.

5. Go back to step 1 and repeat the process, until the refinement is terminated. In practice, the refinement stops when no unlabeled instance $u$ can be found between the $QCS$ and the $CCS$.

Remark

Note that MEGA does not require seeding a query. MEGA presents randomly selected images as the first round of examples. Even if all of the images generated in the first round are irrelevant, MEGA uses the negative images to reduce the set of potentially relevant images substantially (the $k$-DNF that bounds the sampling space can be shrunk substantially by the negative images). Thus, the probability is high that a relevant image will be generated in the next round.

### 2.2 Procedure MEGA

The learner invokes procedure *MEGA* (*the Maximizing Expected Generalization Algorithm*) to select the next $K_\alpha$ unlabeled instances to ask for user feedback. To make sure that an unlabeled instance can be useful, MEGA employs two strategies:

1. Bounding the sample space. MEGA avoids choosing useless unlabeled instances by using the $CCS$ and $QCS$ to delimit the sampling boundary.

2. Maximizing the usefulness of a sample. MEGA chooses an example that shall remove the maximum expected number of disjunctive terms. In other words, MEGA chooses an example that can maximize the expected generalization of the concept.

It may appear that if we pick an example that contradicts to the $QCS$ with more disjunctions, we would have a better chance of eliminating more disjunctive terms. This is, however, not true. In order to be useful to refine $QCS$, an example must be labeled by the user as positive. Unfortunately, an example that contradicts to the $QCS$ with more disjunctions is less likely to be labeled positive. Therefore, there is a tradeoff between choosing an example that has more contradictory terms and choosing an example that is more likely to be labeled positive.

Let $\Psi$ denote the number of disjunctions remaining in the concept. The number of disjunctions that can be eliminated in the current round of sampling, denoted as $\psi$, is between zero and $\Psi$. We can write the probability of eliminating $\psi$ terms as $P_e(\psi)$. In other words, $P_e(\psi)$ is the probability of a sample to be labeled positive, given that the sample contradicts to the $QCS$ with $\psi$ disjunctions. $P_e(\psi)$ is a monotonically decreasing function of $\psi$.

Our MEGA scheme attempts to find the optimal value of $\psi$ that can eliminate the maximum expected number of disjunctive terms given an example. MEGA's objective function can be written as

$$\psi^* = argmax_\psi E(\psi) = \psi \times P_e(\psi). \quad (1)$$

To solve $\psi^*$, we must know $P_e(\psi)$, which can be estimated by the two methods below: probabilistic estimation and empirical estimation.

- Probabilistic estimation. If an example contradicts more disjunctive terms, it is less likely to be labeled positive. For

example, an example contradicting predicate $P_1$ will not be positive only if $P_1$ is in the user's query concept, and an example contradicting both predicates $P_1$ and $P_2$ will not be positive if at least one of either $P_1$ or $P_2$ is in the user's query concept. Let random variable $\Phi_i$ be 1 if $P_i$ is in the concept and 0 otherwise. For simplicity, we assume that the $\Phi_i$'s are iid (independent and identically distributed), and the probability of $\Phi_i$ being 1 is $p$ ($0 < p < 1$). An example contradicting $\psi$ remaining disjunctive terms is marked positive only when none of these $\psi$ terms appears in the user's query concept. This probability is $(1-p)^\psi$. If we substitute $P_e(\psi)$ by $(1-p)^\psi$ on the right-hand side of Equation 1, we get

$$\max E(\psi) = \psi(1-p)^\psi.$$

We can find the optimal $\psi$ value, denoted by $\psi^*$:

$$\psi^* = \Psi,\ if\ \frac{1}{ln\frac{1}{1-p}} > \Psi, \quad \psi^* = \frac{1}{ln\frac{1}{1-p}},\ otherwise.$$

Of course, it may be too strong an assumption that the probability $p$ of all disjunctions is iid. However, we do not need a precise estimation here for the following two reasons:

1. Precise estimation may not be feasible and can be computationally intensive.
2. An approximate estimation is sufficient for bootstrapping. Once the system is up and running for a while and collects enough data, it can empirically estimate $P_e(\psi)$ using its past experience. We discuss this process next.

- Empirical estimation. The concept learner can estimate $P_e(\psi)$ based on its past experience. For each example it presents, it records how many disjunctions the example contradicts with respect to the query concept and whether the example is labeled positive. Once the system collects a sufficient amount of data, we can estimate $P_e(\psi)$ empirically. We then pick the $\psi^*$ that can eliminate the maximum expected number of disjunctive terms.

Again, a reasonable approach to estimate $P_e(\psi)$ is using probabilistic estimation when the system first starts and then switching to empirical estimation when the system has collected sufficient data.

### 2.3 Procedure Vote

There can be two sources of noise:

- Model bias. The target concept may not be in $k$-CNF.
- User errors. A user may label some positive instances negative, and vice versa.

Procedure *Vote* controls the strictness of voting among labeled instances using $\gamma$. The larger the value of $\gamma$ is, the more strict the voting is and therefore the harder it is to eliminate a term. When the noise level is high, we have less confidence on the correctness of user feedback. Thus, we want to be more cautious about eliminating a term. Being more cautious means increasing $\gamma$. Increasing $\gamma$, however, makes the learning process converge more slowly. To learn a concept when noise is present, one has to buy accuracy with time.

## 3 Experiments

We have conducted extensive experiments [2], but here we report just some highlights due to space limitations. We used both synthesized data and real image data in our experiments. For the synthesized data, we generated a dataset with $10,000$ vectors and each vector consists of $M = 10$ attributes: $P_1, P_2, \ldots, P_{10}$. The values of each attribute were generated independently and they were uniformly distributed between 0 and 1. For the image dataset, we used a set of $2,000$ Corel and Web images consisting of fifteen categories: architecture, people, landscape, flowers, and etc. Each image was characterized by an 144-dimension feature vector. (Please see [1] for the specification of our demo system.)

To measure the performance on the synthesized data, we first found all "golden objects" in the database that satisfy the target query concept. Supposing that there were $K$ of them, we then ran an experiment for up to five iterations of relevance feedback to find out how many of these "golden objects" could be obtained when we performed a top-$K$ retrieval after the $i^{th}$ iteration ($i = 1..5$). The ratio of "golden objects" retrieved is called *convergence ratio*.

We tested both MEGA and MARS and compared the results. We chose MARS because it employs a representative IR scheme. To ensure that the results would be objective, we ran each experiment 100 times, each time starting from a randomly selected query point. Due to space limitations, we show just the learning results of the following three concepts:

1. $P_1 \vee P_2$,
2. $(P_1 \vee P_2) \wedge P_3$,
3. $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7 \vee P_8)$,

| *Iteration* | $P_1 \vee P_2$ | | $(P_1 \vee P_2) \wedge P_3$ | | Complex Concept | |
|---|---|---|---|---|---|---|
| *Number* | *MARS* | *MEGA* | *MARS* | *MEGA* | *MARS* | *MEGA* |
| 0 | 0.203 | 0.237 | 0.129 | 0.034 | 0.085 | 0.057 |
| 1 | 0.444 | 0.300 | 0.109 | 0.159 | 0.142 | 0.285 |
| 2 | 0.452 | 1.000 | 0.228 | 0.402 | 0.200 | 0.314 |
| 3 | 0.513 | 1.000 | 0.313 | 1.000 | 0.185 | 0.542 |
| 4 | 0.780 | 1.000 | 0.333 | 1.000 | 0.228 | 0.914 |
| 5 | 0.882 | 1.000 | 0.333 | 1.000 | 0.257 | 0.914 |

Table 1: Learning Results on Synthesized Dataset.

Table 1 shows that in all cases MEGA performs significantly better than MARS in both learning the concepts accurately and converging to the concept quickly. And as the concept gets more and more complex, the performance gap between MEGA and MARS increases. When the query concept is relatively simple as $P_1 \vee P_2$, MARS manages to achieve a $88.2\%$ convergence ratio after five iterations, whereas MEGA learns the concept perfectly after just two iterations. For the more complex concept $(P_1 \vee P_2) \wedge P_3$, MARS can only attain less than $40\%$ convergence ratio after five iterations, whereas MEGA can still learn the concept perfectly just in three iterations. When the complexity of the concept rises to $P_1 \wedge (P_2 \vee P_3) \wedge (P_4 \vee P_5 \vee P_6) \wedge (P_2 \vee P_4 \vee P_7 \vee P_8)$, the performance of MARS struggles to maintain over $25\%$, whereas MEGA reaches $90\%$ after five iterations. These results confirm our intuition (discussed in Section 1.1) that a query refinement

| *Categories* | Iteration 1 | | Iteration 2 | | Iteration 3 | | Iteration 4 | | Iteration 5 | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 | Top 10 | Top 20 |
| Architecture | 0.470 | 0.410 | 0.750 | 0.560 | 0.900 | 0.645 | 0.960 | 0.750 | 0.990 | 0.865 |
| Bears | 0.210 | 0.165 | 0.550 | 0.340 | 0.750 | 0.505 | 0.870 | 0.630 | 0.930 | 0.710 |
| Clouds | 0.340 | 0.210 | 0.750 | 0.635 | 0.990 | 0.940 | 0.980 | 0.980 | 0.950 | 0.970 |
| Elephants | 0.570 | 0.375 | 0.590 | 0.450 | 0.720 | 0.505 | 0.790 | 0.630 | 0.870 | 0.680 |
| Fabrics | 0.410 | 0.285 | 0.620 | 0.440 | 0.860 | 0.595 | 0.900 | 0.755 | 0.910 | 0.855 |
| Fireworks | 0.690 | 0.585 | 1.000 | 0.965 | 1.000 | 1.000 | 1.000 | 0.995 | 1.000 | 1.000 |
| Flowers | 0.330 | 0.205 | 0.750 | 0.525 | 0.960 | 0.720 | 1.000 | 0.850 | 0.990 | 0.960 |
| Food | 0.580 | 0.465 | 0.980 | 0.950 | 1.000 | 0.985 | 1.000 | 0.995 | 1.000 | 1.000 |
| Landscape | 0.270 | 0.200 | 0.550 | 0.365 | 0.730 | 0.500 | 0.850 | 0.615 | 0.890 | 0.715 |
| Objectionable | 0.420 | 0.275 | 0.860 | 0.705 | 1.000 | 0.890 | 1.000 | 0.945 | 1.000 | 0.985 |
| People | 0.360 | 0.240 | 0.550 | 0.350 | 0.740 | 0.470 | 0.810 | 0.570 | 0.940 | 0.695 |
| Textures | 0.400 | 0.300 | 0.850 | 0.675 | 0.970 | 0.835 | 1.000 | 0.945 | 1.000 | 0.960 |
| Tigers | 0.480 | 0.345 | 0.740 | 0.610 | 0.910 | 0.715 | 0.910 | 0.765 | 0.930 | 0.820 |
| Tools | 0.500 | 0.320 | 0.940 | 0.865 | 1.000 | 0.990 | 1.000 | 0.995 | 1.000 | 0.995 |
| Waves | 0.400 | 0.340 | 0.630 | 0.445 | 0.880 | 0.735 | 0.970 | 0.855 | 0.980 | 0.930 |
| Avarage | 0.429 | 0.315 | 0.741 | 0.592 | 0.894 | 0.735 | 0.936 | 0.818 | 0.959 | 0.876 |

Table 2: Experimental Results on Image Dataset.

method that selects samples only in the nearest neighborhood of the query point achieves minimum progress by the definition of the nearest neighborhood. On the other hand, MEGA selects informative samples so that target concepts can be learned exponentially fast [4].

We also conducted experiments on a $2,000$-image dataset. An 144-dimension feature vector is extracted for each image containing information about color histograms, color moments, and textures. We assume query concepts can be modeled in 3-CNF. In each experiment, we began with a set of $20$ randomly generated images for querying user feedback. After each iteration, we evaluated the performance by retrieving top-$K$ images based on the concept we had learned. We recorded the ratio of these images that satisfied the user's concept. We ran each experiment through up to five rounds of relevance feedback, since we deemed that it is not realistic to expect an interactive user to conduct too many rounds of feedback. We ran each experiment $10$ times with different initial starting samples.

Table 2 shows the results for each of the fifteen query concepts, for $K = 10$ and $20$. The table shows that significant improvement on precision is achieved in the first three feedback iterations. For most of the queries, after just three iterations the top-$10$ retrieval achieves $90\%$ precision. For top-$20$ retrieval, the average precision reaches $81.1\%$ after four feedback iterations and surpasses $85\%$ after five iterations.

## 4 Conclusion

We have presented a model of a query-concept learner that combines inductive and active learning. Our experimental results showed that our active learning scheme, MEGA, outperforms traditional sampling schemes significantly in all query scenarios when the number of labeled instances is limited. In addition, we showed that the learner can cope well with model bias and user errors. For additional results please refer to the extended version of this paper [2]. We are currently exploring parallel algorithms for splitting up features to speed up learning time.

Finally, in a separate study we have recently developed a support vector machine active learning algorithm, $\mathsf{SVM}_{Active}$ [9], which can quickly capture a user's query concept by separating relevant images from the irrelevant images with a learned hyperplane in a projected space, usually a very high dimensional one. We believe that MEGA and $\mathsf{SVM}_{Active}$ can work in a complementary way and are studying how these two methods can be effectively combined.

## References


[1] E. Chang, K.-T. Cheng, and L. Chang. Pbir: Perception-based image retrieval. *ACM Sigmod (Exhibition)*, May 2001.

[2] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.ps*, November 2000.

[3] I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.

[4] Y. Freund, H. S. Seung, E. Shamir, and N. Tishby. Selective sampling using the query by committee algorithm. *Machine Learning*, 28:133–168, 1997.

[5] K. S. Jones and P. W. (Editors). *Readings in Information Retrieval*. Morgan Kaufman, July 1997.

[6] T. Michell. *Machine Learning*. McGraw Hill, 1997.

[7] K. Porkaew, K. Chakrabarti, and S. Mehrotra. Query refinement for multimedia similarity retrieval in mars. *Proceedings of ACM Multimedia*, November 1999.

[8] Y. Rui, T. S. Huang, and S.-F. Chang. Image retrieval: Current techniques, promising directions and open issues. *Journal of Visual Communication and Image Representation*, March 1999.

[9] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Technical Report http://www-db.stanford.edu/∼echang/svmactive.ps*, March 2001.

[10] L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.